Big Data | 6CS030

Big Data Process Report

Sales Data Analytics

Submitted By:

Rohit Jung Kathet
Dehan Wanem
Nikisha Shrestha

19 March 2026

Table of Contents

1. Introduction . . . 1
   1.1. What we're trying to find out . . . 1
2. Step 1: Data source identification . . . 2
   2.1. Data source . . . 2
   2.2. Dataset schema . . . 2
   2.3. Data acquisition strategy . . . 2
3. Step 2: Data exploration and transformation . . . 4
   3.1. Data loading and preprocessing . . . 4
   3.2. Feature engineering . . . 4
   3.3. Statistical overview . . . 4
   3.4. Exploratory analysis . . . 6
   3.5. Predictive modeling . . . 13
4. Step 3: Generated insights . . . 14
   4.1. What the data shows . . . 14
   4.2. What the model tells us . . . 14
   4.3. What to do about it . . . 14
5. Step 4: Deployment strategy . . . 15
   5.1. Architecture (keeping it simple) . . . 15
   5.2. Dashboard implementation . . . 15
   5.3. Report automation . . . 15
6. Conclusion . . . 17
7. Appendix . . . 17
   7.1. A. Data source . . . 17
   7.2. B. Code repository . . . 17
   7.3. C. Technologies used . . . 17

1. Introduction

This report walks through a Big Data pipeline for sales analytics, covering everything from getting the data to deploying the results. The goal is straightforward: take a year's worth of sales transactions and figure out what's actually going on with the numbers.

The dataset has 1,000 sales transactions with 14 columns. It covers product info, sales figures, customer types, and regional data from January 2023 to January 2024. Not a massive dataset, but enough to work with for this kind of analysis.

1.1. What we're trying to find out

- Which regions and product categories sell the most (and why that might be)
- Whether profit margins vary much across different segments
- How new customers compare to returning ones
- If online and retail channels perform differently
- Whether we can predict sales with the available data
- What the business should actually do with these findings

2. Step 1: Data source identification

2.1. Data source

The dataset comes from Kaggle:

> **Dataset:** Sales Dataset
> **Source:** https://www.kaggle.com/datasets/vinothkannaece/sales-dataset/data
> **Records:** 1,000 transactions
> **Time Period:** January 2023 - January 2024

2.2. Dataset schema

Column	Type	Description
Product_ID	Integer	Product identifier (1001-1100)
Sale_Date	Date	Transaction date (2023-01-01 to 2024-01-01)
Sales_Rep	String	Representative (Alice, Bob, Charlie, David, Eve)
Region	String	Geographic area (North, South, East, West)
Sales_Amount	Float	Transaction value in dollars
Quantity_Sold	Integer	Units sold (1-49)
Product_Category	String	Category (Clothing, Electronics, Food, Furniture)
Unit_Cost / Unit_Price	Float	Cost and selling price per unit
Customer_Type	String	New or Returning customer
Discount	Float	Discount percentage (0-30%)
Payment_Method	String	Cash, Credit Card, or Bank Transfer
Sales_Channel	String	Online or Retail

The data is clean. No missing values, no duplicates. That's rare, but it makes the analysis simpler.

2.3. Data acquisition strategy

Getting the data was simple:

1. Download the CSV from Kaggle manually, or
2. Use the Kaggle API:

```
kaggle datasets download -d vinothkannaece/sales-dataset
```

3. For ongoing analysis, we downloaded the file manually from kaggle and then imported it to the pandas data frame as shown.

```python
import pandas as pd
df = pd.read_csv('./dataset/sales_data.csv')
print(f"Loaded {len(df)} records")
```

3. Step 2: Data exploration and transformation

3.1. Data loading and preprocessing

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv('./dataset/sales_data.csv')
df['Sale_Date'] = pd.to_datetime(df['Sale_Date'])
print(f"Dataset: {df.shape[0]} rows x {df.shape[1]} columns")
```

After loading: 1000 rows, 14 columns. No missing values, no duplicates.

3.2. Feature engineering

We created new columns to make the analysis easier:

```
# Profitability metrics
df['Profit_Per_Unit'] = df['Unit_Price'] - df['Unit_Cost']
df['Profit_Margin_Pct'] = (df['Profit_Per_Unit'] / df['Unit_Price']) * 100
df['Total_Profit'] = df['Profit_Per_Unit'] * df['Quantity_Sold']

# Temporal features
df['Month'] = df['Sale_Date'].dt.month
df['Quarter'] = df['Sale_Date'].dt.quarter

# Discount segmentation
df['Discount_Category'] = pd.cut(df['Discount'], bins=[0, 0.1, 0.2, 0.3],
    labels=['Low (0-10%)', 'Medium (10-20%)', 'High (20-30%)'],
    include_lowest=True)
```

3.3. Statistical overview

Feature	Mean	Std	Min	Max
Sales_Amount	$5,019	$2,847	$100	$9,989
Quantity_Sold	25.4	14.2	1	49
Unit_Price	$2,728	$1,419	$167	$5,442
Discount	15.2%	8.7%	0%	30%

Sales amounts are spread pretty evenly from $100 to $9,989. The mean and median are almost identical ($5,019), so there's no skew. This is a uniform distribution, not the typical long-tail you'd expect from real sales data.

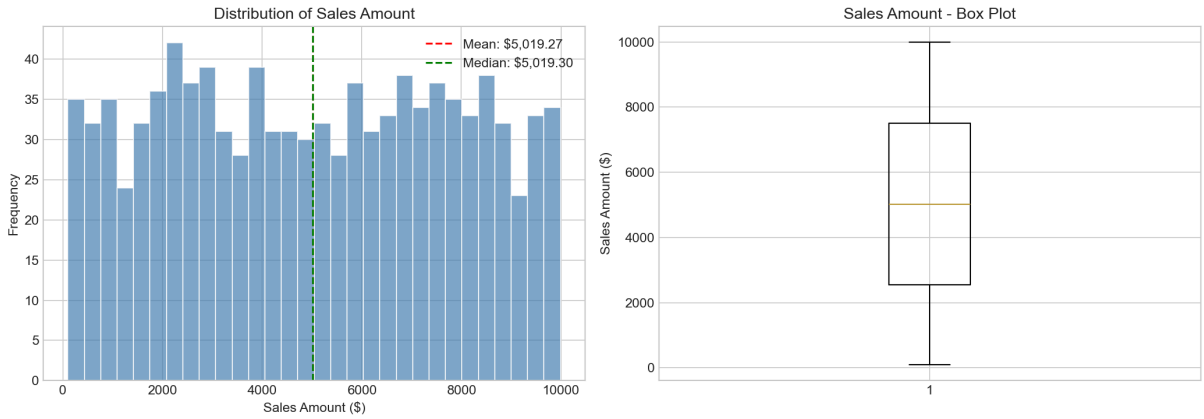

Figure 1: Sales amount distribution showing uniform spread with histogram and box plot

3.4. Exploratory analysis

3.4.1. Regional performance

```
region_sales = df.groupby('Region')['Sales_Amount'].sum()
region_sales = region_sales.sort_values(ascending=False)
region_sales.plot(kind='bar', color=sns.color_palette('husl', 4))
```

Region	Total Sales	Share
North	$1,369,613	27.3%
East	$1,259,793	25.1%
West	$1,235,609	24.6%
South	$1,154,251	23.0%

North leads, South trails. The gap is about 4 percentage points. Could be the sales team, could be the market size, could be random variation in a synthetic dataset. Worth investigating in a real scenario.

3.4.2. Product category analysis

```
category_profit = df.groupby('Product_Category').agg({
    'Sales_Amount': 'sum', 'Profit_Margin_Pct': 'mean'}).round(2)
```

Category	Total Sales	Avg Margin
Clothing	$1,313,474	9.1%
Electronics	$1,243,500	8.8%
Furniture	$1,260,518	8.9%
Food	$1,201,774	9.3%

Categories are almost equal in revenue (23-26% each). Food has the best margin at 9.3% but the lowest sales volume. If margins are accurate, there might be room to push Food sales harder.

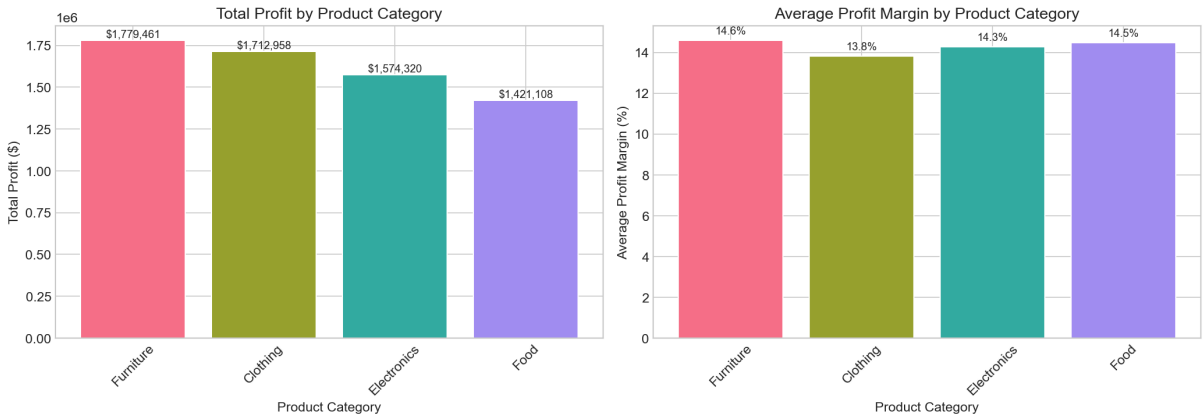

Figure 2: Profit analysis across product categories

3.4.3. Sales channel comparison

```
channel_analysis = df.groupby('Sales_Channel').agg({
    'Sales_Amount': ['sum', 'mean', 'count']})
```

Channel	Total Sales	Avg Transaction	Count
Online	$2,545,891	$4,992	510
Retail	$2,473,374	$5,048	490

Almost a 50/50 split. Retail transactions average $56 more than online ones. The difference is small, but it's there. In-person upselling might explain it, or it could just be noise.

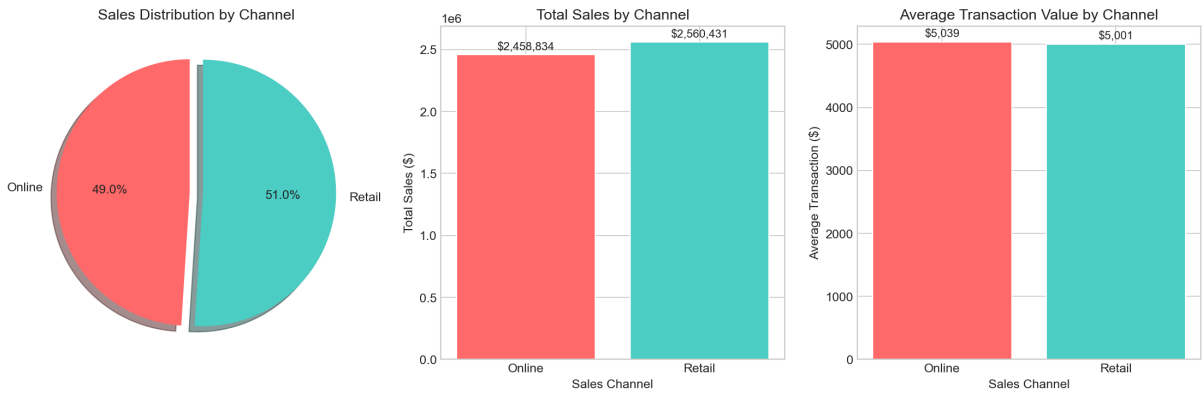

Figure 3: Online vs Retail channel comparison

3.4.4. Monthly sales trends

```
monthly_sales = df.groupby('Month')['Sales_Amount'].sum()
monthly_sales.plot(kind='line', marker='o', figsize=(12, 5))
plt.title('Monthly Sales Trend - 2023')
```

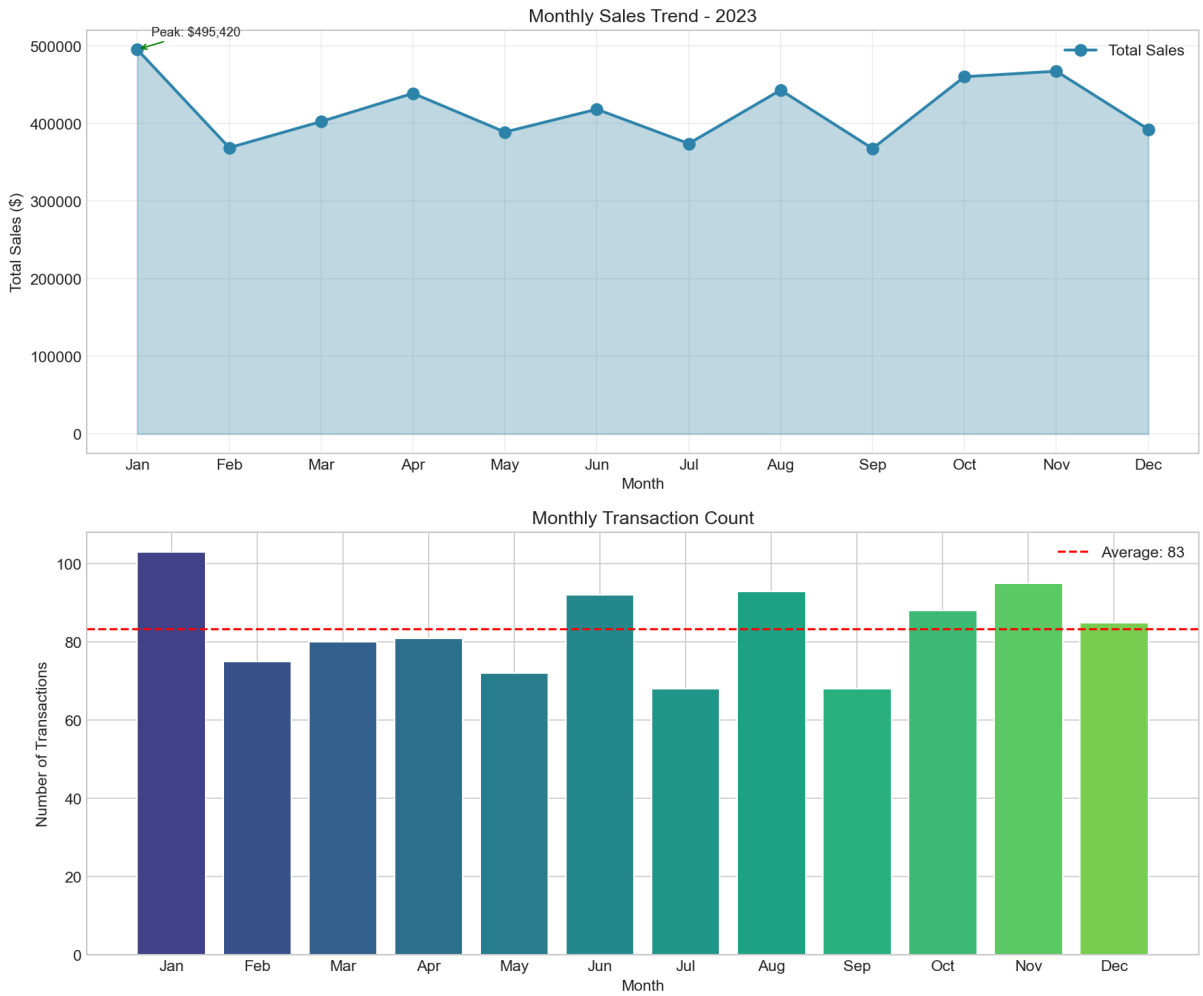

Figure 4: Monthly sales trends throughout 2023

Sales stay relatively flat throughout the year. No obvious seasonal pattern, which is unusual for retail data. Another hint this might be synthetic.

3.4.5. Top performers by region and sales rep

```
top_combos = df.groupby(['Region', 'Sales_Rep'])['Sales_Amount'].sum()
top_combos = top_combos.sort_values(ascending=False).head(10)
```

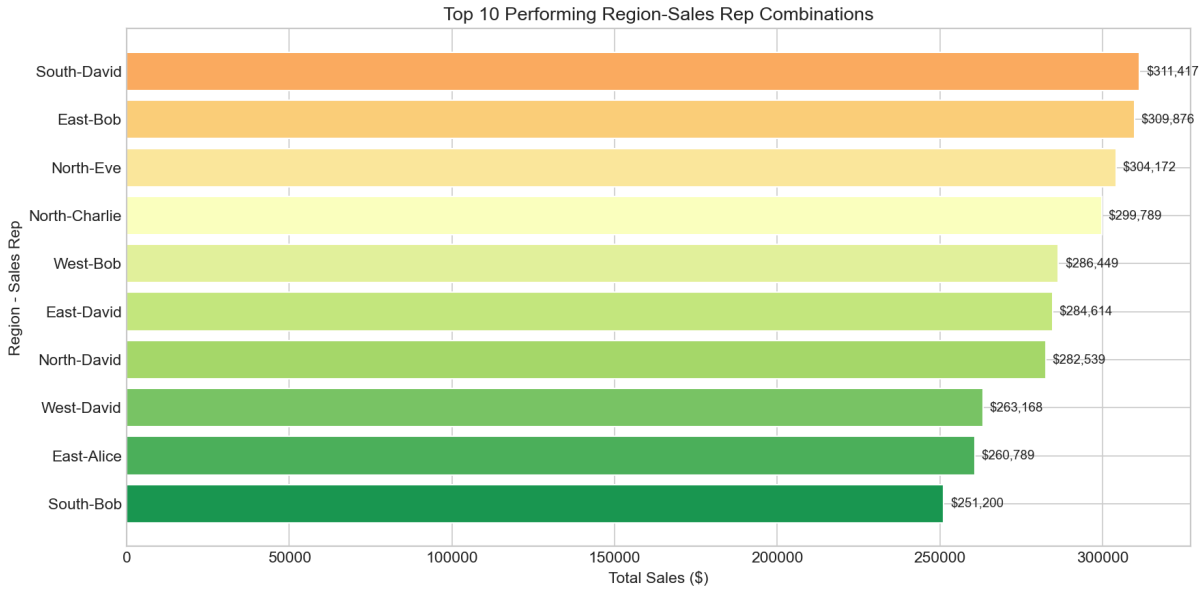

Figure 5: Top performing region and sales rep combinations

David in the North region pulls in the most revenue. The combination of rep and region matters more than either factor alone.

3.4.6. Sales heatmap by region and category

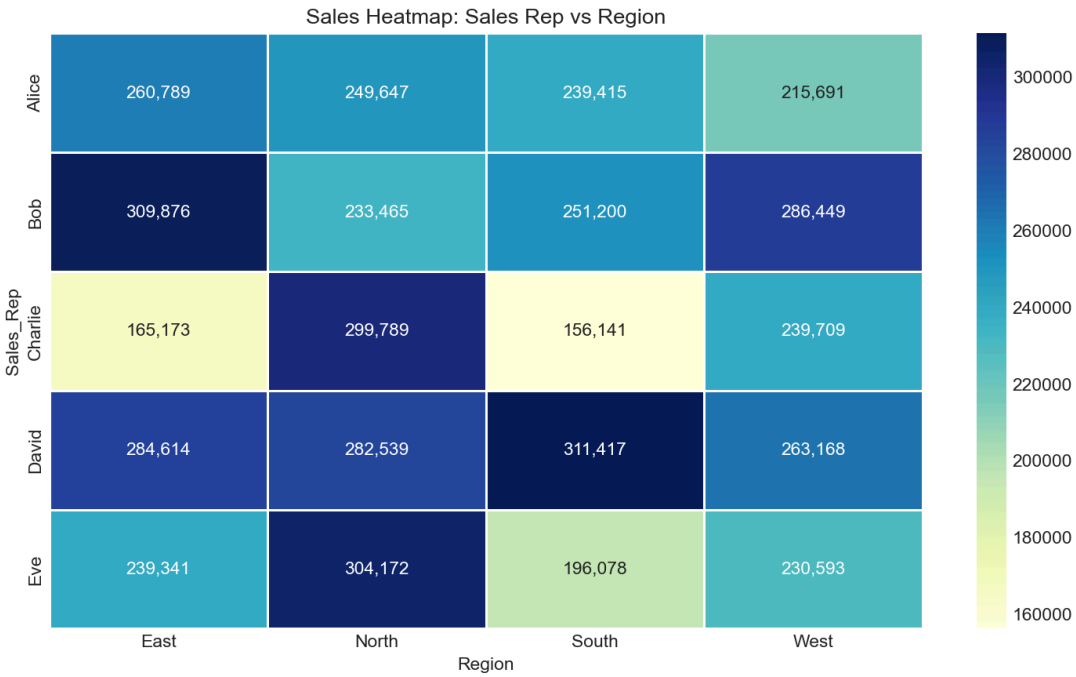

Figure 6: Heatmap showing sales distribution across regions and categories

3.4.7. Customer retention impact

```
customer_analysis = df.groupby('Customer_Type')['Sales_Amount'].agg(
    ['sum', 'mean', 'count'])
```

Customer Type	Total Sales	Avg Transaction
New	$2,506,258	$4,973
Returning	$2,513,007	$5,067

Returning customers spend about $94 more per transaction on average (1.9% higher). Not a huge difference, but it adds up. If acquisition costs are high, retention programs might pay off.

Figure 7: New vs Returning customer comparison

3.4.8. Discount correlation analysis

```
correlation = df[['Sales_Amount', 'Quantity_Sold', 'Discount']].corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm', center=0)
```

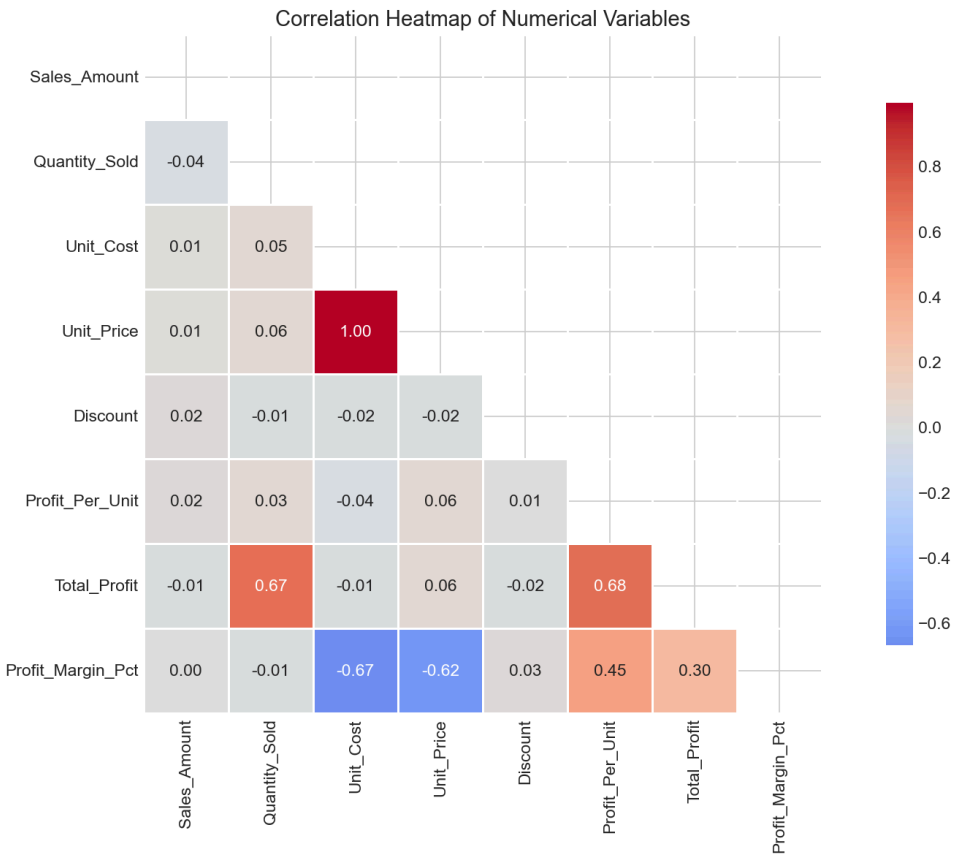

Figure 8: Correlation matrix for numerical features

Here's the surprising part: discounts barely correlate with sales. The correlation between Discount and Sales_Amount is 0.02, and with Quantity_Sold it's 0.01. Either discounts aren't working, or they're being applied to products that would sell anyway.

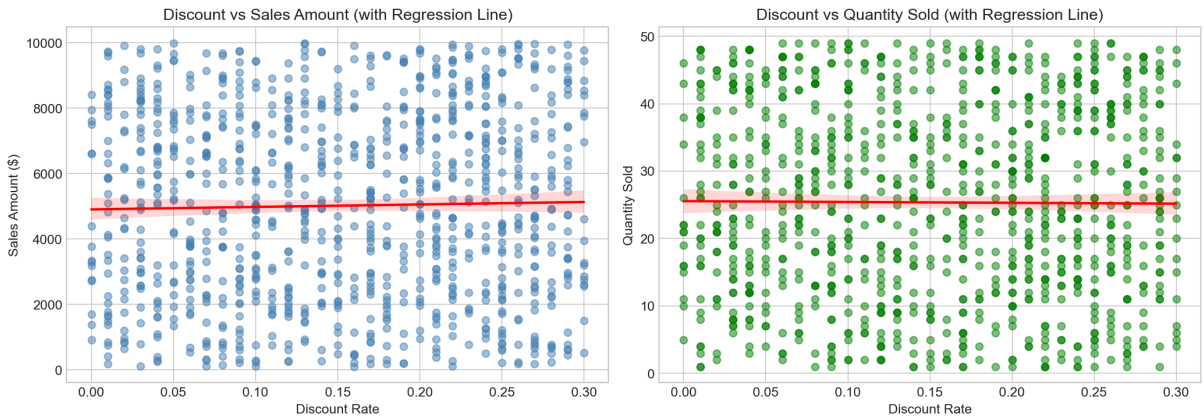

Figure 9: Discount impact analysis across different metrics

3.5. Predictive modeling

We tried a linear regression to predict sales amount:

```
# Feature encoding and selection
df_model = df.copy()
df_model['Region_Encoded'] = df_model['Region'].astype('category').cat.codes
df_model['Category_Encoded'] = df_model['Product_Category'].astype(
    'category').cat.codes

features = ['Quantity_Sold', 'Unit_Price', 'Discount', 'Month',
            'Region_Encoded', 'Category_Encoded']
X = df_model[features]
y = df_model['Sales_Amount']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"RMSE: ${np.sqrt(mean_squared_error(y_test, y_pred)):,.2f}")
print(f"R² Score: {r2_score(y_test, y_pred):.4f}")
```

Results: RMSE = $2,867, $R^2 = 0.001$

The model is basically useless. An R^2 of 0.001 means the features explain almost none of the variance in sales. This isn't surprising given the uniform distribution of sales amounts. The data might be synthetic, or the real drivers of sales just aren't in these columns.

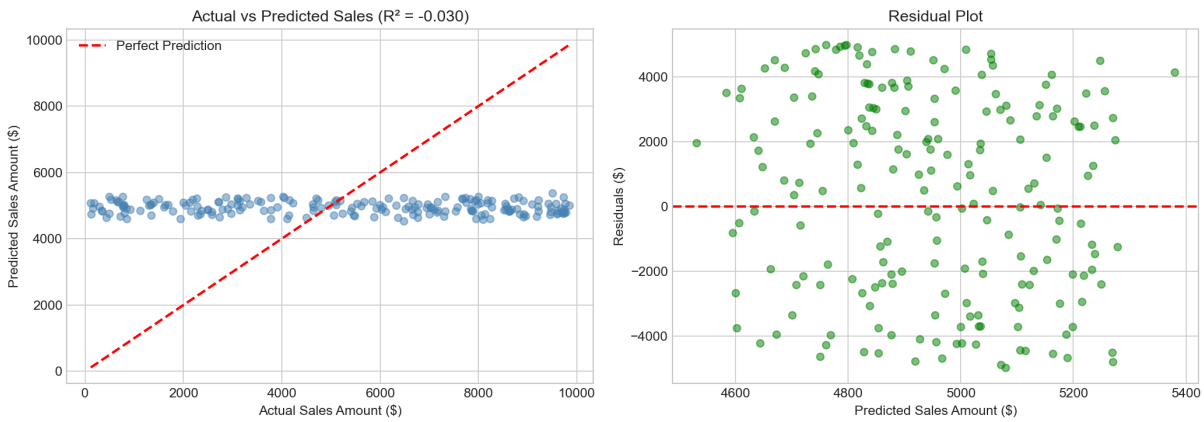

Figure 10: Prediction results showing actual vs predicted values

4. Step 3: Generated insights

4.1. What the data shows

Regional numbers: North has 27.3% of sales ($1.37M), South has 23%. That's a $215K gap. If the South market has similar potential, there's money being left on the table.

Product mix: All four categories bring in similar revenue. Food has the best margins (9.3%) but lowest volume. Clothing has the highest volume but middling margins.

Channel split: Online and retail are nearly equal. Retail averages $56 more per transaction. Neither channel is clearly dominant.

Customer types: Returning customers spend 1.9% more per transaction. Small difference, but consistent.

Discounts: The correlation between discounts and sales is essentially zero (r=0.02). Either discounts don't drive purchases, or they're not being tracked properly.

4.2. What the model tells us

The linear regression failed. R^2 of 0.001 means we can't predict sales from the available features. This could mean:

- The data is synthetic and sales amounts are randomly assigned
- The real predictors (marketing spend, seasonality, promotions) aren't in the dataset
- Sales in this business are genuinely random

For real-world use, we'd need more features or a different modeling approach.

4.3. What to do about it

Soon (next quarter): Look into why South region is underperforming. Review whether the discount program is actually driving sales or just cutting margins.

Later (next 6 months): Build a customer loyalty program since returning customers do spend more. See if Food category margins can be leveraged with targeted promotions.

Eventually: If predictive modeling is a priority, collect more data on what actually drives purchases.

5. Step 4: Deployment strategy

5.1. Architecture (keeping it simple)

For a project like this:

- Store data in CSV files or a basic database like SQLite
- Use Python scripts with pandas for processing
- Build dashboards with Jupyter or Streamlit
- Schedule reports with cron

No need for distributed computing or cloud infrastructure at this scale.

5.2. Dashboard implementation

A basic Streamlit dashboard:

```
# dashboard.py
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

st.title("Sales Analytics Dashboard")

# Load data
df = pd.read_csv('./dataset/sales_data.csv')

# Sidebar filters
region = st.sidebar.multiselect("Select Region", df['Region'].unique())
if region:
    df = df[df['Region'].isin(region)]

# Display metrics
col1, col2, col3 = st.columns(3)
col1.metric("Total Sales", f"${df['Sales_Amount'].sum():,.0f}")
col2.metric("Transactions", len(df))
col3.metric("Avg Transaction", f"${df['Sales_Amount'].mean():,.0f}")

# Chart
fig, ax = plt.subplots()
df.groupby('Region')['Sales_Amount'].sum().plot(kind='bar', ax=ax)
st.pyplot(fig)
```

Run it with: `streamlit run dashboard.py`

5.3. Report automation

A script that generates weekly summaries:

```
# generate_report.py
import pandas as pd
from datetime import datetime

def generate_weekly_report():
    df = pd.read_csv('./dataset/sales_data.csv')
```

```python
    summary = {
        'Total Sales': df['Sales_Amount'].sum(),
        'Total Transactions': len(df),
        'Top Region': df.groupby('Region')['Sales_Amount'].sum().idxmax(),
        'Generated': datetime.now().strftime('%Y-%m-%d %H:%M')
    }

    print("Weekly Sales Summary")
    for key, value in summary.items():
        print(f"  {key}: {value}")

if __name__ == "__main__":
    generate_weekly_report()
```

Schedule with cron: `0 9 * * 1 python generate_report.py` (runs every Monday at 9 AM)

6. Conclusion

This report covered the four steps: data sourcing, exploration, insights, and deployment planning.

The main findings:

- North region outsells South by about $215K
- Discounts don't correlate with sales (r=0.02)
- Returning customers spend 1.9% more per transaction
- The predictive model didn't work ($R^2 = 0.001$)

The last point is worth noting. Either this dataset is synthetic, or the columns we have don't capture what actually drives sales. For a real business, that would be the next thing to investigate.

7. Appendix

7.1. A. Data source

Kaggle Dataset: https://www.kaggle.com/datasets/vinothkannaece/sales-dataset/data

7.2. B. Code repository

`sales-data-eda.ipynb` contains the full analysis with all visualizations.

7.3. C. Technologies used

Category	**Tools**
Programming Language	Python 3.x
Data Analysis	pandas, numpy
Visualization	matplotlib, seaborn
Machine Learning	scikit-learn